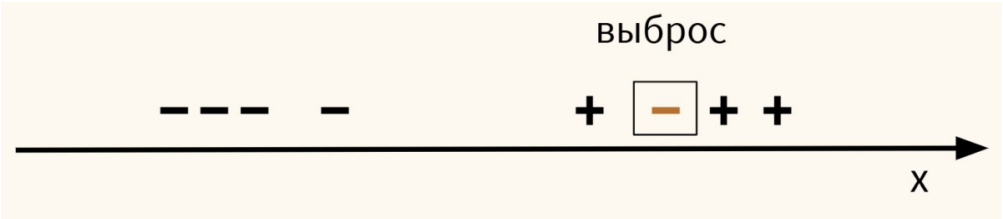
Метод опорных векторов. Часть 2

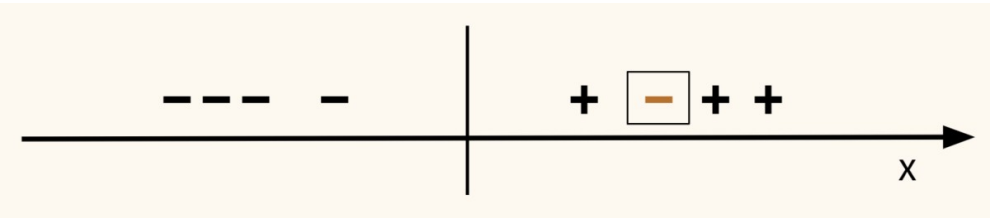
SVM с мягким зазором

Основная идея

Вернемся к примеру с единственным признаком, однако теперь представим, что в данных присутствуют выбросы и из-за этого они **линейно неразделимы** (linearly non-separable).



При этом если позволить алгоритму ошибиться с классификацией одного наблюдения, мы получим модель, которая (возможно) будет хорошо предсказывать новые данные.



Такой алгоритм называется **методом опорных векторов с мягким зазором** (soft-margin SVM).

Допуская большее или меньшее количество ошибочно классифицируемых наблюдений мы можем либо увеличивать точность алгоритма на обучающей выборке, либо повышать его обобщающую способность.

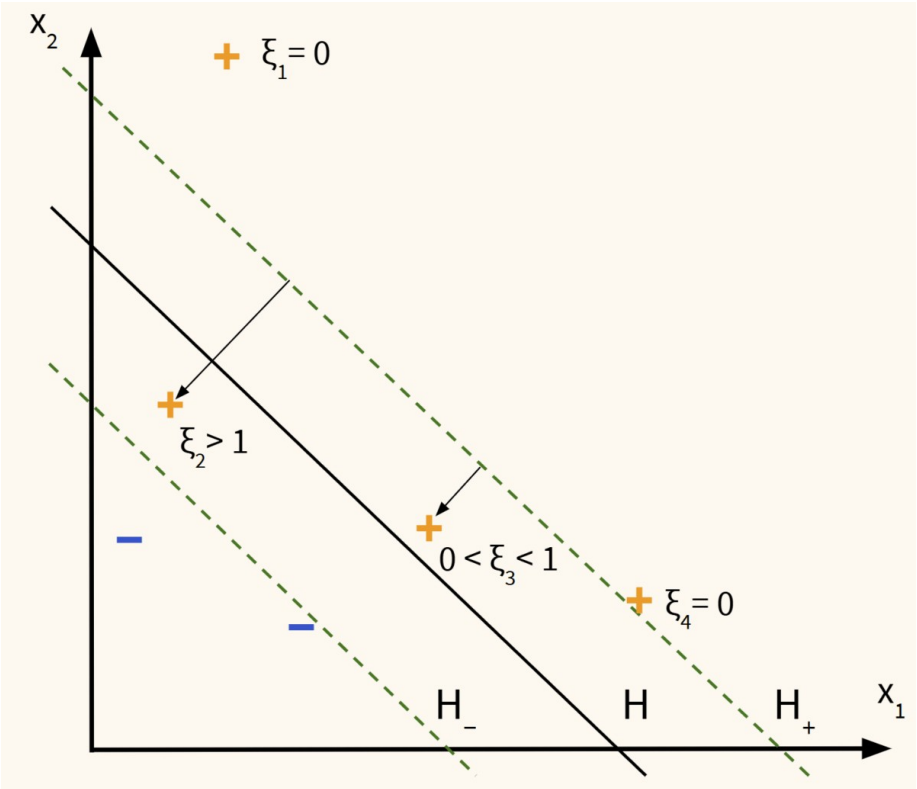
Можно сказать, что таким образом обеспечивается компромисс между смещением и разбросом (bias-variance tradeoff).

Оценка ошибочной классификации

Для того чтобы обучить такой алгоритм для *каждого* наблюдения введем **переменные** ξ_i, где $i = 1, \ldots, n$ (slack variable)

$$\xi_i = 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b), \quad \xi_i \geq 0,$$

которые показывают, насколько алгоритм ошибся при классификации объекта.



На схеме выше для положительных объектов:

- если наблюдение находится на «своей» стороне или непосредственно на «своем» зазоре H_+, то ξ, и, в частности, ξ_1 и ξ_4 равны нулю;
- если объект находится между H и H_+, то, например, $0 < \xi_3 < 1$;
- если наблюдение оказывается на «неправильной» стороне, то $\xi_2 > 1$.

Аналогичное рассуждение можно провести и для отрицательных объектов.

Таким образом, просуммировав все ξ_i мы получим оценку того, какое количество объектов было классифицировано неправильно $\xi_i > 1$ или не идеально $0 < \xi_i < 1$.

Это количество нам бы хотелось минимизировать.

Задача оптимизации

Основная задача

Сформулируем основную задачу оптимизации.

$$\min_{\mathbf{w},\mathbf{b},\xi} \left(\frac{1}{2}||\mathbf{w}||^2 + C\sum_{i=1}^{n} \xi_i \right)$$
$$y_i \left(\mathbf{w}^\top \mathbf{x}_i + b \right) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

Обратим внимание на два момента:

- во-первых, $1 - \xi_i$ будет меньше единицы только в том случае, если классификатор ошибается (а также допускает объекты между гиперплоскостью и «своим» зазором);
- кроме того, гиперпараметр C определяет значимость такой ошибки, $C\xi_i$:

- если C невелико, например, $C = 0{,}1$, то количество ошибочно классифицированных объектов может быть довольно большим (широкий коридор);
- если наоборот велико, $C = 100$, то алгоритм постарается не допускать ошибочной классификации (узкий коридор);
- при $C = \infty$ получается метод опорных векторов с жестким зазором.

Примечание. Слагаемое $C\sum_{i=1}^{n}\xi_i$ можно считать L1-регуляризатором функции потерь.

Двойственная задача

Введем **функцию Лагранжа**.

$$\begin{aligned}\mathcal{L}(\mathbf{w}, b, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\beta}) = \quad & \tfrac{1}{2}\mathbf{w}^\top\mathbf{w} + C\sum_{i=1}^{N}\xi_i \\ & -\sum_{i=1}^{N}\alpha_i[y_i(\mathbf{w}^\top\mathbf{x}_i + b) - 1 + \xi_i] - \sum_{i=1}^{N}\beta_i\xi_i\ ,\end{aligned}$$

где $\boldsymbol{\alpha}, \boldsymbol{\beta}$ — множители Лагранжа, и поставим задачу ее **максимизации**

$$\max_{\boldsymbol{\alpha},\boldsymbol{\beta}:\boldsymbol{\alpha}\geq 0,\boldsymbol{\beta}\geq 0}\mathcal{L}(\mathbf{w}, b, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\beta})$$

Найдем производные относительно $\mathbf{w}, b, \boldsymbol{\xi}$ и приравняем их к нулю.

$$\begin{aligned}\frac{\partial\mathcal{L}}{\partial\mathbf{w}} &= \mathbf{w} - \sum_{i=1}^{N}\alpha_i y_i\mathbf{x}_i = 0 \\ \mathbf{w} &= \sum_{i=1}^{N}\alpha_i y_i\mathbf{x}_i\end{aligned}$$

$$\begin{aligned}\frac{\partial\mathcal{L}}{\partial b} = -\sum_{i=1}^{N}\alpha_i y_i &= 0 \\ \sum_{i=1}^{N}\alpha_i y_i &= 0\end{aligned}$$

$$\frac{\partial\mathcal{L}}{\partial\boldsymbol{\xi}} = C - \alpha_i - \beta_i = 0$$
$$\beta_i = C - \alpha_i$$

Выразим ограничения для множителей α_i, β_i через ограничение для α_i.

$$\beta_i \geq 0 \implies C - \alpha_i \geq 0 \implies \alpha_i \leq C$$
$$\alpha_i \geq 0, \alpha_i \leq C \implies 0 \leq \alpha_i \leq C$$

Подставив полученные значения обратно в $\mathcal{L}$ и преобразав выражение, получим

$$\begin{aligned}
\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) &= \tfrac{1}{2}\sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j + C\sum_{i=1}^N \xi_i \\
&\quad + \sum_{i=1}^N \alpha_i - \sum_{i=1}^N \alpha_i \xi_i \\
&\quad - \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j - b\sum_{i=1}^N \alpha_i y_i \\
&\quad - C\sum_{i=1}^N \xi_i + \sum_{i=1}^N \alpha_i \xi_i \\
&= \sum_{i=1}^N \alpha_i - \tfrac{1}{2}\sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j
\end{aligned}$$

$$\max_{\boldsymbol{\alpha}} \mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha})$$

с ограничениями $\sum_{i=1}^N \alpha_i y_i = 0$ и $0 \leq \alpha_i \leq C$.

Отдельно рассмотрим **условия дополняющей нежесткости**.

$$\begin{aligned}
\alpha_i^* [y_i(\mathbf{w}^{*\top}\mathbf{x}_i + b^*) - 1 + \xi_i^*] &= 0 \\
\beta_i^* \xi_i^* &= 0
\end{aligned}$$

Исходя из первого условия, если $\alpha_i^* > 0$, то для того, чтобы произведение было равно нулю, $y_i(\mathbf{w}^{*\top}\mathbf{x}_i + b^*) - 1 + \xi_i^*$ должно быть равно нулю. Как следствие, так как $\xi_i^* \geq 0$,

$$y_i(\mathbf{w}^{*\top}\mathbf{x}_i + b^*) - 1 \leq 0$$

Одновременно, если $\beta_i^* > 0$ или, что то же самое, $\alpha_i^* < C$, в силу $\alpha_i^* + \beta_i^* = C$, то $\xi_i^* = 0$ и, поскольку по основному условию осуществимости $y_i(\mathbf{w}^{*\top}\mathbf{x}_i + b^*) \geq 1 - \xi_i^*$, получаем

$$y_i(\mathbf{w}^{*\top}\mathbf{x}_i + b^*) - 1 \geq 0$$

Таким образом,

$$\begin{aligned} \alpha_i^* < C \quad &\Longrightarrow \quad y_i(\mathbf{w}^{*\top}\mathbf{x}_i + b^*) - 1 \geq 0 \\ \alpha_i^* > 0 \quad &\Longrightarrow \quad y_i(\mathbf{w}^{*\top}\mathbf{x}_i + b^*) - 1 \leq 0 \end{aligned}$$

Или, что то же самое,

$$\begin{aligned} \alpha_i^* = 0 \quad &\Longrightarrow \quad y_i(\mathbf{w}^{*\top}\mathbf{x}_i + b^*) - 1 \geq 0 \\ 0 < \alpha_i^* < C \quad &\Longrightarrow \quad y_i(\mathbf{w}^{*\top}\mathbf{x}_i + b^*) - 1 = 0 \\ \alpha_i^* = C \quad &\Longrightarrow \quad y_i(\mathbf{w}^{*\top}\mathbf{x}_i + b^*) - 1 \leq 0 \end{aligned}$$

Другими словами, в методе опорных векторов с мягким зазором наблюдения с $\alpha_i^* \neq 0$ будут включать как опорные объекты, так и ошибочно классифицированные наблюдения, для которых $\xi_i > 0$.

Таким образом, задача сводится к **минимизации** (выпуклого) квадратичного функционала

$$\min_{\boldsymbol{\alpha}} \left(\frac{1}{2} \boldsymbol{\alpha}^\top H \boldsymbol{\alpha} - \mathbf{1}^\top \boldsymbol{\alpha} \right)$$

с ограничениями

$$\begin{aligned} &\mathbf{0} \leq -\boldsymbol{\alpha} \leq \mathbf{C} \\ &\mathbf{y}^\top \boldsymbol{\alpha} = 0, \end{aligned}$$

где единственным отличием от метода опорных векторов с жестким зазором будет дополнительное ограничение $-\boldsymbol{\alpha} \leq \mathbf{C}$.

Код на Питоне

Библиотека CVXOPT

С точки зрения формата библиотеки CVXOPT нам необходимо внести измерения в матрицу G и вектор h. Новая матрица $G_{2n \times n}$ будет

состоять из двух вертикально соединенных диагональных матриц размерности $n \times n$ со значениями -1 и 1 на главных диагоналях соответственно.

К вектору $h_{2n\times 1}$ мы добавим n значений C. Для $n = 2$ G и h примут вид

$$G = \begin{bmatrix} -1 & 0 \\ 0 & -1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad h = \begin{bmatrix} 0 \\ 0 \\ C \\ C \end{bmatrix}$$

Класс SoftSVM

Продолжим в том же блокноте

```
class SoftSVM:

  def __init__(self, C=None):
    self.support_alphas = None
    self.support_vectors = None
    self.support_vector_labels = None
    self.W = None
    self.intercept = None
    self.C = C

  def fit(self, X, y):
    n_samples, n_features = np.shape(X)

    K = np.zeros((n_samples, n_samples))
    for i in range(n_samples):
      for j in range(n_samples):
        K[i,j] = np.dot(X[i], X[j])

    P = cvxopt.matrix(np.outer(y,y) * K)
    q = cvxopt.matrix(np.ones(n_samples) * -1)
    A = cvxopt.matrix(y, (1, n_samples), tc='d')
    b = cvxopt.matrix(0.0)
```

```
    if not self.C:
      G = cvxopt.matrix(np.identity(n_samples) * -1)
      h = cvxopt.matrix(np.zeros(n_samples))
    else:
      G_max = np.identity(n_samples) * -1
      G_min = np.identity(n_samples)
      G = cvxopt.matrix(np.vstack((G_max, G_min)))
      h_max = cvxopt.matrix(np.zeros(n_samples))
      h_min = cvxopt.matrix(np.ones(n_samples) * self.C)
      h = cvxopt.matrix(np.vstack((h_max, h_min)))

    solution = cvxopt.solvers.qp(P, q, G, h, A, b)
    alphas = np.ravel(solution['x'])

    idx = alphas > 1e-5
    self.support_alphas = alphas[idx]
    self.support_vectors = X[idx]
    self.support_vector_labels = y[idx]

    self.W = np.dot((self.support_vector_labels * self.support_alpl
    self.intercept = np.mean(self.support_vector_labels - np.dot(s

    return self

  def predict(self, X):

    return np.sign(np.dot(X, self.W) + self.intercept)
```

Линейно неразделимые данные

Изменим класс *одного* наблюдения в данных, чтобы сделать их линейно неразделимыми.

```
def get_non_separable_data():
  X_train = np.array([[-3.5, -1], [-3, 0], [-3, 1], [-2.7, -1.3],
                      [-1, -2.5], [0, -3], [0, 1], [0, 2.5], [1, 2
                      [2.1, 0.2], [2.3, 1], [2.8, 1.8], [2.2, 2.8]
  y_train = np.array([-1, -1, -1, -1, -1, -1, -1, -1, 1, -1, 1, 1,
  X_test = np.array([[0, 0], [1, -3]])
  y_test = np.array([1, -1])

  return X_train, y_train, X_test, y_test
```

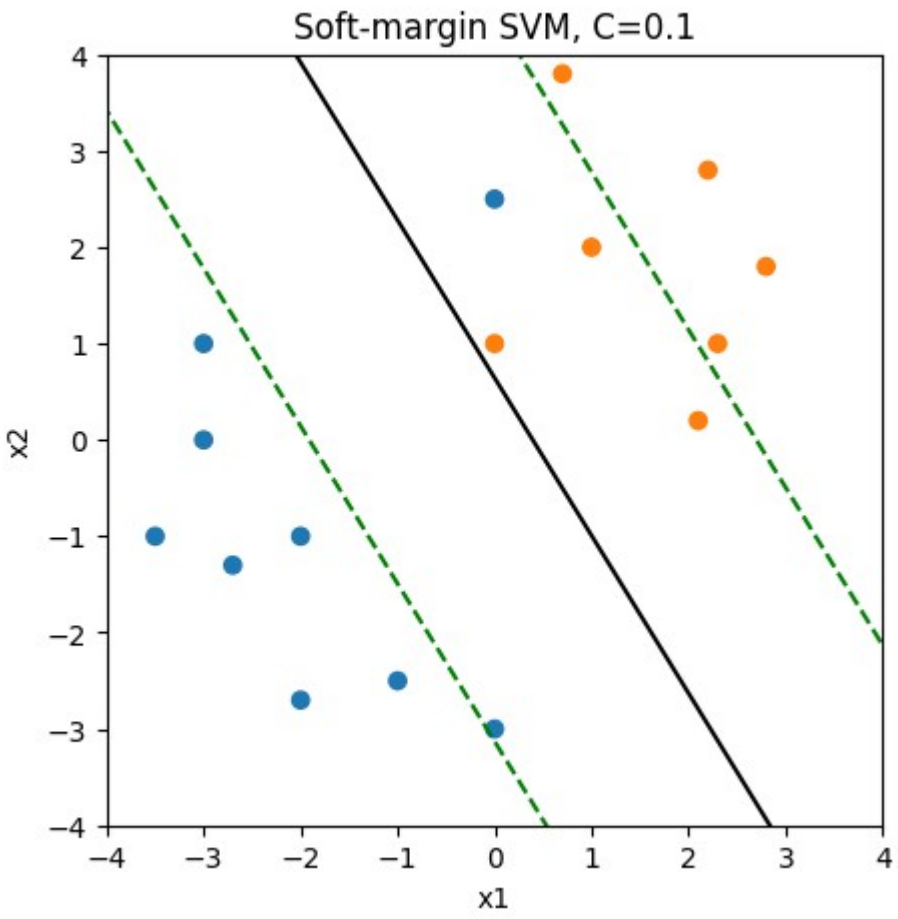
Создадим обучающую выборку.

```
X_train, y_train, _, _ = get_non_separable_data()
```

Обучение модели

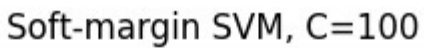
Вначале обучим модель с параметром $C = 0{,}1$.

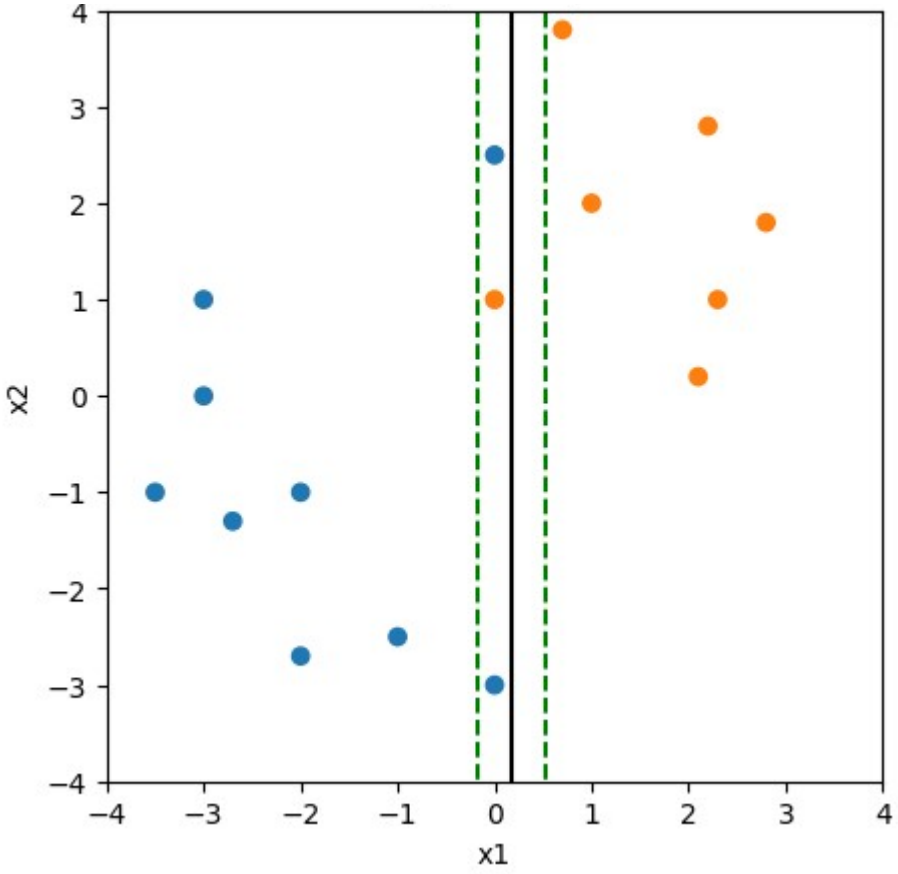
```
soft_svm = SoftSVM(C=0.1).fit(X_train, y_train)
plot_data(X_train, y_train, title='Soft-margin SVM, C=0.1',
          w=soft_svm.W, b=soft_svm.intercept)
```



Теперь изменим параметр на $C = 100$.

```
soft_svm = SoftSVM(C=100).fit(X_train, y_train)
plot_data(X_train, y_train, title='Soft-margin SVM, C=100',
          w=soft_svm.W, b=soft_svm.intercept)
```





Ожидаемо, с ростом значимости слагаемого регуляризации алгоритм старается не допустить ошибочной классификации.

SVM и метод градиентного спуска

Hinge loss

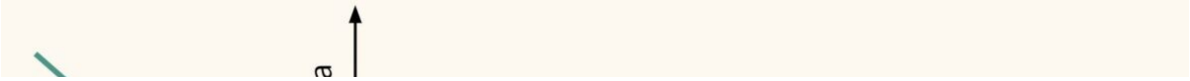
Обратим внимание, что данное выше определение ξ_i

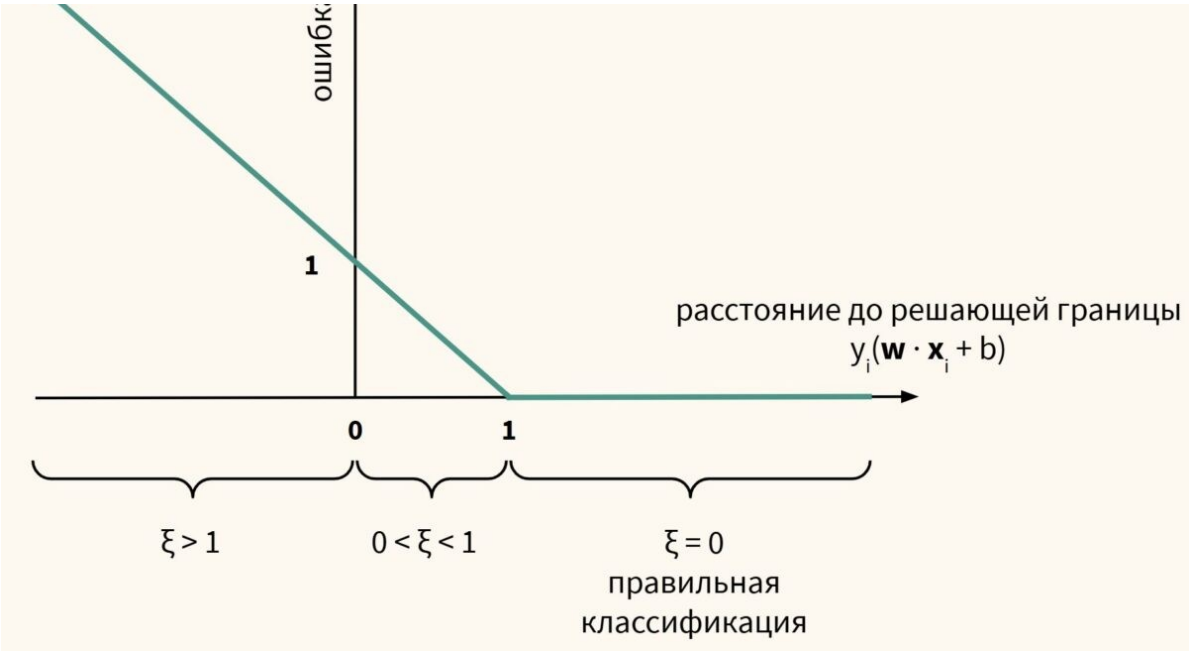
$$\xi_i = 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b), \quad \xi_i \geq 0$$

можно записать и так

$$\xi_i = \max\big(\big(1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)\big), 0\big),$$

т.е. представить в виде функции hinge loss





Тогда общую функцию потерь можно записать как

$$\min_{\mathbf{w},b} \left(\frac{1}{N} \sum_{i=1}^{N} \underbrace{\max\left(\left(1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)\right), 0\right)}_{\text{hinge loss}} + \underbrace{\frac{\lambda}{2} ||\mathbf{w}||^2}_{\text{regularizer}} \right),$$

где, по сравнению с решаемой через квадратичное программирование задачей, мы поменяли местами слагаемые и теперь:

- hinge loss является основной функцией потерь; а
- слагаемое $||\mathbf{w}||^2$ — L2 регуляризатор с коэффициентом $\frac{\lambda}{2}$.

Субградиентный спуск

Найдем градиент функции hinge loss. Вначале для удобства внесем коэффициент b в вектор параметров $\mathbf{w}$, добавив в данные столбец из единиц.

Теперь разложим функцию потерь $J(\mathbf{w})$ на два возможных сценария.

$$\begin{cases} J(\mathbf{w}) = \frac{\lambda}{2}||\mathbf{w}||^2 & \text{если} \quad y_i(\mathbf{w}^\top \mathbf{x}_i) \geq 1 \\ J(\mathbf{w}) = \frac{\lambda}{2}||\mathbf{w}||^2 + 1 - y_i(\mathbf{w}^\top \mathbf{x}_i) & \text{если} \quad y_i(\mathbf{w}^\top \mathbf{x}_i) < 1 \end{cases}$$

Примечание. Поскольку в точке $y_i(\mathbf{w}^\top \mathbf{x}_i) = 1$ градиент не определен, используем один из возможных субградиентов.

Найдем (суб)градиент для каждого из сценариев.

$$\begin{cases} \frac{\partial J}{\partial \mathbf{w}} = \lambda \mathbf{w} \\ \frac{\partial J}{\partial \mathbf{w}} = \lambda \mathbf{w} - y_i \mathbf{x}_i \end{cases}$$

При этом гиперпараметры λ и C связаны следующим отношением

$$C = \frac{1}{\lambda N} \quad \Longrightarrow \quad \lambda = \frac{1}{CN}$$

Код на Питоне

Приведем код на Питоне. Будем использовать стохастический градиентный спуск, то есть обновлять веса на основе одного объекта.

```
class GradientSVM:

  def __init__(self, learning_rate=1e-3, lambda_param=1e-2, epochs=
    self.lr = learning_rate
    self.lambda_param = lambda_param
    self.epochs = epochs
    self.W = None
    self.loss_history = []

  def _compute_hinge_loss(self, x, y, idx):

    return max(0, 1-(y[idx] * np.dot(x, self.W)))

  def _compute_gradients(self, x, y, idx):
    # найдем градиенты для каждого сценария
    if (y[idx] * (np.dot(x, self.W))) >= 1:
      dw = self.lambda_param * self.W
    else:
```

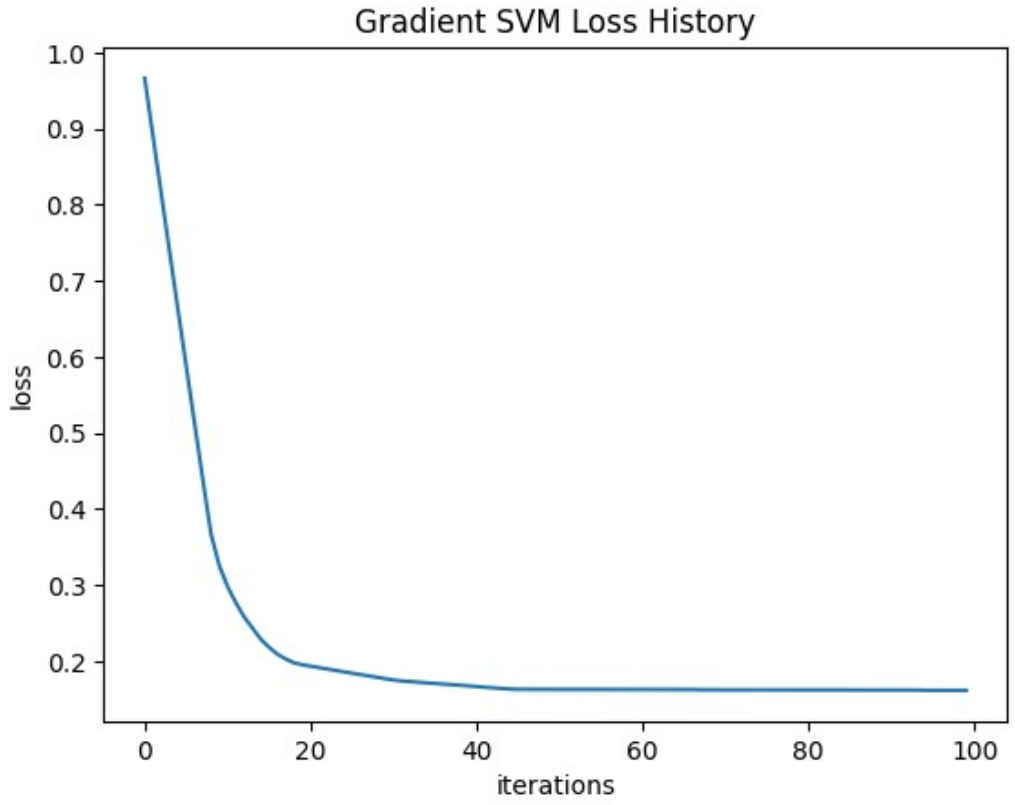
```
      dw = self.lambda_param * self.W - y[idx] * x

    return dw

  def fit(self, X, y):
    # добавим столбец из единиц
    X = np.c_[np.ones((len(X), 1)), X]
    # инициализируем веса с учетом дополнительного признака
    self.W = np.zeros(X.shape[1])

    # на каждой эпохе
    for _ in range(self.epochs):
      loss = 0
      # для каждого объекта
      for idx, x in enumerate(X):
        # рассчитаем и учтем уровень ошибки
        loss += self._compute_hinge_loss(x, y, idx)
        # найдем градиент
        dw = self._compute_gradients(x, y, idx)
        # обновим веса
        self.W -= self.lr * dw
      # для каждой эпохи рассчитаем и
      # запишем средний уровень ошибки
      self.loss_history.append(loss/len(X))

    return self

  def predict(self, X):
    X = np.c_[np.ones((len(X), 1)), X]

    return np.sign(np.dot(X, self.W))
```

Вначале обучим алгоритм с параметром $C = 0{,}1$ или $\lambda = \frac{1}{0{,}1 \cdot 16} = 0{,}625$.

```
X_train, y_train, X_test, y_test = get_non_separable_data()
```

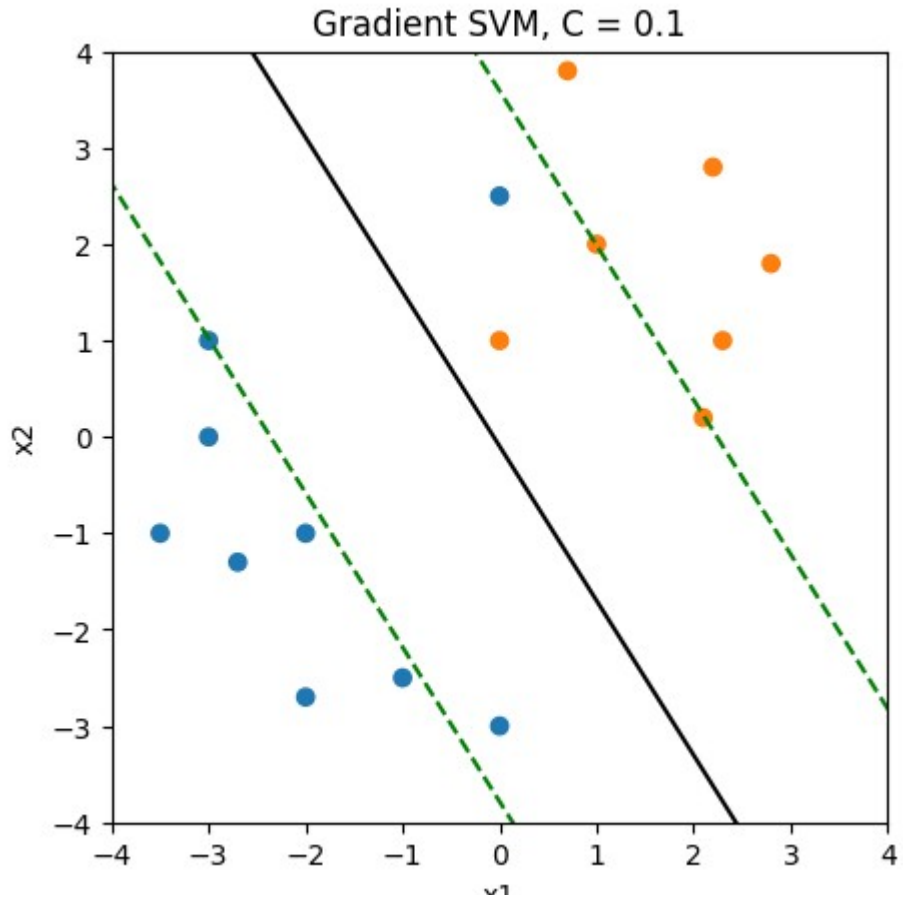
```
C1=.1
lambda1 = 1/(2 * C1 * X_train.shape[0])
grad_svm = GradientSVM(lambda_param=lambda1).fit(X_train, y_train)
```

```
plt.plot(grad_svm.loss_history[:100])
plt.title('Gradient SVM Loss History')
plt.xlabel('iterations')
plt.ylabel('loss')
plt.show()
```



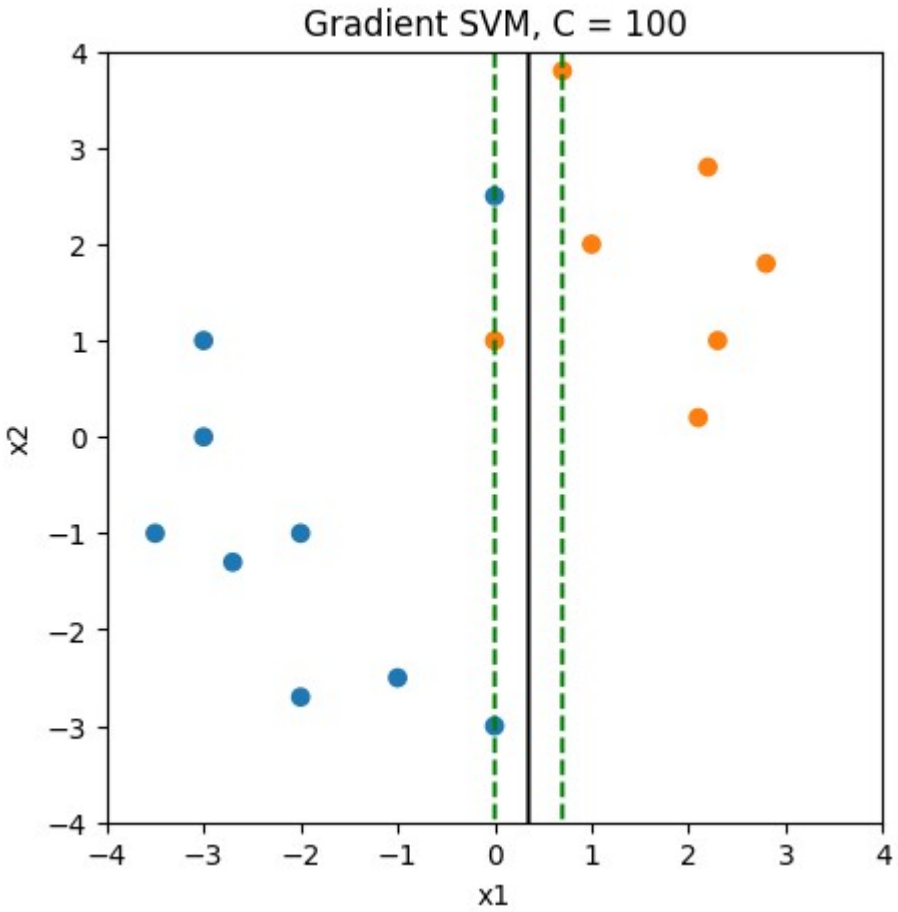
Посмотрим на гиперплоскость.

```
plot_data(X_train, y_train, title='Gradient SVM, C = 0.1',
          w=grad_svm.W[1:], b=grad_svm.W[0])
```



Теперь обучим модель с параметром $C = 100$ или $\lambda = \frac{1}{100 \cdot 16} = 0{,}000625$.

```
C2 = 100
lambda2 = 1/(2 * C2 * X_train.shape[0])
grad_svm2 = GradientSVM(lambda_param=lambda2).fit(X_train, y_train
plot_data(X_train, y_train, title='Gradient SVM, C = 100',
          w=grad_svm2.W[1:], b=grad_svm2.W[0])
```



Примечания:

- в библиотеке Sklearn классификатор SVM с оптимизацией методом градиентного спуска реализован в классе SGDClassifier с параметром loss = ‘hinge’;
- для оптимизации SVM методом субградиентного спуска с уменьшающимся размером шага предложен алгоритм Pegasos.